

Verilog 數位電路設計 — 18 週自學課程

對應課綱：鄭信源《Verilog 硬體描述語言數位電路設計實務》全部用 OSS CAD Suite (免費開源) ，不需要買 FPGA 板也能跑到最後一週。

怎麼使用這份課程

每次開始工作：

1. 雙擊 `env.bat` (在 VERI 資料夾裡)
2. 提示字元變成 `[OSS CAD Suite]` 就代表環境好了
3. `cd` 進當週資料夾，先讀 `README.md`，再跑範例

跑一個範例，就是這四行：

<code>iverilog -o sim.out 01_gates.v 01_gates_tb.v</code>	← 編譯
<code>echo %ERRORLEVEL%</code>	← 印 0 才是成功
<code>vvp sim.out</code>	← 模擬，產生 <code>wave.vcd</code>
<code>gtkwave 01_gates.gtkw</code>	← 開波形

把 `01_gates` 換成你要跑的範例名稱就好。每個範例完整的四行都列在 [HOWTO.md](#)。

只想看文字不開波形 → 省略最後那行。想自己拉訊號 → 最後一行改成 `gtkwave wave.vcd`。唯一的例外是第 7 週的 `04_netlist`，編譯要多加參數 (`HOWTO.md` 有寫)。

進度總表

週次	主題	資料夾	狀態
00	語法入門 (Python 對照) + 暖身計數器	week00_warmup/	✓ 已建好
01	設計的基本概念	week01_basics/	✓ 已建好
02	架構、資料型別與運算子	week02_types_ops/	✓ 已建好
03	組合與循序邏輯	week03_comb_seq/	✓ 已建好
04	函式 / 程序與屬性 / 組態	week04_func_param/	✓ 已建好
05	功能模擬	week05_simulation/	✓ 已建好
06	狀態機設計	week06_fsm/	✓ 已建好
07	時序模擬	week07_timing_sim/	✓ 已建好
08	期中考 (複習 + 綜合題)	week08_midterm/	✓ 已建好
09	記憶體設計	week09_memory/	✓ 已建好
10	階層式設計	week10_hierarchy/	📋 待建
11	設計合成	week11_synthesis/	📋 待建
12	管線式處理	week12_pipeline/	📋 待建
13-14	ASIC 設計與實作	week13_asic/	📋 待建
15	FPGA 應用實習	week15_fpga/	📋 待建
16	期末考 (專題)	week16_final/	📋 待建
17-18	佈局繞線與時序分析	week17_pnr/	📋 待建

各週詳細規劃

第 0 週 — 語法入門

給誰	寫軟體出身 (Python / C / Java) 、對 Verilog 語法完全陌生的人
內容	SYNTAX.md : 14 節的 Python↔Verilog 對照 —— 註解、分號、數字寫法、 <code>{}</code> 、 <code>wire / reg</code> 、 <code>for</code> 、 <code>if / case</code> 、 <code>module</code> 、 <code>always</code> 、 <code>=</code> vs <code><=</code> 、 <code>function</code> 、 <code>\$display</code> 、Python 沒有的東西
範例	<code>02_syntax</code> 語法練習場 (把 <code>{}</code> 四種用法和 <code>for</code> 實際跑一次) · <code>01_counter</code> 暖身計數器
重點	★ 三個思維轉換：所有東西同時發生 / 每個訊號都有固定位元寬度 / 縮排沒有意義

第 1 週 — 設計的基本概念

觀念	硬體 vs 軟體思維、同時 vs 依序、 <code>module</code> 骨架、 <code>assign</code> 、實例化
範例	<code>01_gates</code> 四個基本閘 · <code>02_half_adder</code> 半加器 · <code>03_full_adder</code> 全加器
波形重點	組合邏輯「輸入一變輸出立刻變」的樣子；GTKWave 左邊的階層樹
練習	加 NAND 輸出 · 自己組 2-bit 加法器

第 2 週 — 架構、資料型別與運算子

觀念	<code>wire</code> vs <code>reg</code> 的真正差別 · 四值邏輯 <code>0 1 x z</code> · 數字表示法 <code>4'b1010</code> <code>8'hFF</code> · 向量與位元切片 <code>a[3:1]</code> · 串接 <code>{a,b}</code> · 縮減運算子 <code>&a</code> · 位移 <code><<</code> <code>>></code> · 三元 <code>?:</code> · 有號 vs 無號
範例	<code>01_vectors</code> 向量與切片 · <code>02_operators</code> 五類運算子 · <code>03_concat</code> 串接與符號延伸 · <code>04_mux</code> 三元運算子多工器 · <code>05_xz</code> <code>x</code> 與 <code>z</code> · <code>06_operators_all</code> 運算子全集 (查表用)
波形重點	★ 親眼看到 <code>x</code> (未知) 和 <code>z</code> (高阻抗) 長什麼樣 —— 這是初學最常見的除錯線索
練習	用位移和串接做一個 4-bit 循環左移 · 判斷一組數的奇偶 (用縮減 XOR)

第 3 週 — 組合與循序邏輯

觀念	<code>always @(*)</code> 組合 vs <code>always @(posedge clk)</code> 循序 · ★ 阻塞 <code>=</code> vs 非阻塞 <code><=</code> ★ · <code>if / case</code> · latch 陷阱 (漏寫 <code>else</code> 會怎樣) · D 型正反器 · 移位暫存器
範例	<code>01_case_mux</code> case 多工器與解碼器 · <code>02_latch_trap</code> 故意做出 latch · <code>03_dff</code> 同步/非同步 reset · <code>04_blocking_vs_non</code> <code>=</code> vs <code><=</code> · <code>05_shift_reg</code> 移位暫存器
波形重點	★★ <code>04</code> 是整週精華：同樣三行程式碼， <code><=</code> 走 3 拍、 <code>=</code> 只走 1 拍，一看就懂
練習	修好 <code>02_latch_trap</code> · 環形計數器 · 可預載上下數計數器

第 4 週 — 函式 / 程序與屬性 / 組態

觀念	<code>function</code> vs <code>task</code> · <code>parameter</code> / <code>localparam</code> · 模組參數化 <code>#(...)</code> · <code>generate</code> 迴圈 · 巨集 <code>`define</code> / <code>`include</code> / <code>`ifdef</code>
範例	<code>01_function</code> 三個實用函式 · <code>02_param_counter</code> 三種參數同時跑 · <code>03_generate</code> generate 做 N-bit 加法器 · <code>04_macro</code> 條件編譯
波形重點	同一個 <code>param_counter</code> · 參數改成 4/8/16 位元 · 波形自動跟著變
練習	<code>function</code> 版半加器 · 參數化格雷碼計數器 · generate 版優先權編碼器

第 5 週 — 功能模擬

觀念	<code>`timescale</code> 到底在做什麼 · <code>\$display</code> <code>\$write</code> <code>\$monitor</code> <code>\$strobe</code> 差在哪 · <code>\$time</code> <code>\$finish</code> <code>\$stop</code> · 自我檢查測試平台 · 隨機測試 <code>\$random</code> · 測試涵蓋率概念 · <code>\$readmemh</code> 讀測試向量
範例	<code>01_display_family</code> 四種印法對照 · <code>02_self_check</code> 2548 組自動比對 · <code>03_timescale</code> 時間單位 · <code>04_random_test</code> 亂數測試抓出藏起來的 bug
波形重點	時間單位改變時波形的橫軸怎麼跟著變
練習	修好 <code>04</code> 裡的 bug · 幫第 4 週的加法器寫窮舉測試

第 6 週 — 狀態機設計

觀念	Moore vs Mealy · 狀態圖怎麼畫 · 三段式寫法 (業界標準) · 狀態編碼 (binary / one-hot / gray) · 預設狀態與 default 的重要性
範例	01_traffic_moore 紅綠燈 · 02_seq_detect 抓 1011 (Moore/Mealy 並排) · 03_debounce 按鍵去彈跳 · 04_vending 投幣販賣機
波形重點	★ 把 state 拉到波形上 · 右鍵 → Data Format → Decimal 直接看狀態編號 ; 03 對照 btn_raw 抖動 vs btn_clean 乾淨
練習	紅綠燈加行人按鈕 · 偵測 110 · 販賣機改規格

第 7 週 — 時序模擬

觀念	功能模擬 vs 時序模擬 · 闌延遲 · setup / hold 時間 · 時脈偏移 · 亞穩態 · 關鍵路徑 · 閘級模擬 (用綜合後的網表再模擬一次)
範例	01_gate_delay 闌延遲 · 02_glitch 毛刺 · 03_cdc_sync 雙 FF 同步器 · 04_netlist ★ Yosys 綜合成 iCE40 網表後用原廠時序再模擬一次 · 跟 RTL 疊起來比
波形重點	★★ 04 是重頭戲 : 放大到 ns 等級 · 看 sum_gate 在進位傳遞時連跳好幾個中途錯值 · 而 sum_rtl 一步到位
練習	改延遲值 · 用正反器過濾毛刺 · 改大設計重新綜合看毛刺變化

第 8 週 — 期中考

內容	前 7 週重點複習 · 常見錯誤清單 · 3 道綜合題 (含解答與測試平台)
題目	① 01_seg7 BCD 七段解碼器 (16 分) ② 02_updown 可預載上下數計數器 (256 分) ③ 03_pattern_fsm 密碼鎖狀態機 (17 分)
形式	每題附空白的設計檔和寫好的測試平台 —— 你填設計 · 測試自動判分

第 9 週 — 記憶體設計

觀念	ROM vs RAM · 同步 vs 非同步讀取 · 單埠 / 雙埠 / 偽雙埠 · 讀寫衝突 · FIFO 與空滿判斷 · <code>\$readmemh</code> 初始化 · FPGA 的 Block RAM 推斷 (怎麼寫才會被綜合成 BRAM)
範例	<code>01_rom</code> ROM 與同步/非同步讀取 · <code>02_ram_sp</code> 單埠 RAM · <code>03_ram_dp</code> 雙埠 RAM · <code>04_fifo</code> 完整 FIFO 含空滿判斷
波形重點	寫入與讀出差一個時脈的現象 ; FIFO 快滿時 <code>full</code> 旗標跳起來的那一刻
練習	改 ROM 內容 · FIFO 加 <code>almost_full/empty</code> · 非同步 FIFO (難)

第 10 週 — 階層式設計

觀念	由上而下 vs 由下而上 · 介面設計 · 命名規範 · 參數往下傳 · 多檔案專案怎麼組織 · 跨模組除錯技巧
範例	<code>01_alu_hier</code> 4-bit ALU 拆成加法器 / 邏輯單元 / 多工器三個子模組 · <code>02_datapath</code> 資料路徑 + 控制器分離 · <code>03_top_integration</code> 把前幾週做的東西整合成一個系統
波形重點	在 GTKWave 樹狀圖中層層往下鑽 · 追一個訊號穿過三層模組
練習	把你的 ALU 改成 8-bit (只改參數 · 不改邏輯)

第 11 週 — 設計合成

觀念	RTL → 邏輯閘的過程 · 可合成 vs 不可合成語法 (哪些只能模擬) · 綜合後面積報告怎麼看 · 資源共享與最佳化 · 為什麼 <code>initial</code> 和 <code>#delay</code> 不能合成
工具	★ 開始用 Yosys
範例	<code>01_synth_basic</code> 綜合第 1 週的加法器看報告 · <code>02_unsynth</code> 一堆不能合成的寫法示範 · <code>03_area_compare</code> 三種寫法做同一件事 · 比面積 · <code>04_show_netlist</code> 把綜合結果畫成電路圖
重點 產出	★ 你會親眼看到自己的程式碼變成「幾個 LUT、幾個 FF」的數字
練習	想辦法把你的 ALU 面積縮小 20%

第 12 週 — 管線式處理

觀念	延遲 (latency) vs 吞吐量 (throughput) · 為什麼切管線可以拉高時脈 · 管線暫存器 · 資料危障 · 管線平衡
範例	<code>01_mult_comb</code> 單週期乘法器 · <code>02_mult_pipe3</code> 切成三級管線 · <code>03_compare</code> 兩者的吞吐量與最高時脈比較 (用 Yosys + icetime 實測)
波形 重點	★ 管線的「填充」與「排空」階段；同時有三筆資料在不同級中前進
練習	把 FIR 濾波器切成管線

第 13–14 週 — ASIC 設計與實作

觀念	ASIC vs FPGA 全流程對照 · 標準元件庫 · 邏輯合成 → 佈局規劃 → 繞線 → DRC/LVS · 靜態時序分析 (STA) · 功耗分析 · 為什麼一顆晶片要幾千萬 · 良率與測試
範例	<code>01_generic_synth</code> 綜合成通用邏輯閘網表 · <code>02_liberty_map</code> 映射到標準元件庫 · <code>03_sta_concept</code> 時序路徑分析 · <code>04_formal</code> ★ 用 <code>sby</code> 做形式驗證 · 數學證明你的設計正確
重點 產出	一份完整的「從 RTL 到網表」的流程紀錄

第 15 週 — FPGA 應用實習

觀念	FPGA 內部結構複習 (LUT / FF / 繞線開關) · 完整流程 Yosys → nextpnr → icepack · 約束檔 (腳位對應) · 時脈管理 · 除彈跳與同步
工具	★ nextpnr-ice40 + icepack
範例	01_blink LED 一秒閃一次 · 02_pwm 呼吸燈 · 03_button 按鍵計數 · 04_seg7 七段顯示器 · 05_uart_tx 序列埠送資料
重點 產出	★ 真的產生出 bitstream 檔案 (就算沒有板子，檔案是真的)

第 16 週 — 期末考 (專題)

內容	三選一的完整專題，含需求規格、測試平台、評分標準
題目	① UART 收發器 (含鮑率產生器) ② 簡易 8-bit CPU (4 個指令) ③ VGA 訊號產生器

第 17–18 週 — 佈局繞線與時序分析

觀念	佈局繞線在解什麼問題 · 時序約束怎麼寫 · 最高時脈怎麼算出來的 · 資源使用率報告 · 時序收斂技巧 · 面積 vs 速度取捨
工具	★ nextpnr (佈局繞線) + icetime (靜態時序分析)
範例	01_pnr_flow 完整跑一次 · 02_timing_report 讀懂時序報告 · 03_optimize 同一設計三種寫法比最高時脈 · 04_utilization 資源使用率分析
重點 產出	★ 一份你自己設計的電路的真實時序報告：這個設計最快能跑幾 MHz

建議節奏

- 一週一個資料夾，先讀 README → 跑範例看波形 → 做練習
- 練習不會就先跳過，但範例一定要跑過並看懂波形
- 每週結尾有「檢核清單」，勾不完不要硬往前
- 卡住就回頭翻 [explain.md](#) (逐行講解) 和 [run.md](#) (操作流程)

資料夾結構

```
VERI/
├─ env.bat           ← 每次工作先雙擊這個
├─ HOWTO.md         ← ★ 每個範例的完整指令
├─ COURSE.md        ← 你正在看的這份（課程規劃）
├─ explain.md       ← 三個檔案的逐行講解
├─ run.md           ← 手動操作流程筆記
├─
├─ week00_warmup/   ← 你已經做完的計數器
├─ week01_basics/   ← 第 1 週
├─ week02_types_ops/
├─ ...
└─ _archive/        ← 舊的一鍵腳本，備用
```